Trinity College – Hartford
Computer Science Department
CPSC-375 – High-Performance Computing

Design and Implementation of a Distributed HPC Cluster

Cluster Name: *Tricolor*

Team 7
Liu Neptali Restrepo Sanabria
liu.restreposanabria@trincoll.edu

Instructor: Peter Yoon, Ph.D.

May 3, 2026

Contents

1 Introduction **2**

2 System Architecture **2**
2.1 Memory Models: From UMA to Distributed NUMA . . . 2
2.2 Hardware Components . . . 3
2.3 Hardware Specifications . . . 3

3 Implementation **3**
3.1 Operating System Installation . . . 3
3.2 Operational Modes . . . 4

4 Networking and Configuration **4**
4.1 Static IP Assignment . . . 4
4.2 Passwordless SSH . . . 4
4.3 NFS Shared Filesystem . . . 4

5 Software Stack **5**

6 Testing and Verification **5**
6.1 Network Connectivity . . . 5
6.2 MPI Hostname Test . . . 5
6.3 MPI Hello World Programme . . . 6
6.4 NFS Verification . . . 6

7 Benchmarking **7**
7.1 HPL Overview . . . 7
7.2 Benchmarking Strategy . . . 7
7.3 HPL Build Configuration . . . 7
7.4 Phase 1: Strong Scaling . . . 7
7.4.1 1-Node (P=2, Q=2) . . . 7
7.4.2 2-Node (P=2, Q=4) . . . 8
7.4.3 3-Node (P=3, Q=4) . . . 8
7.5 Phase 2: Parameter Tuning . . . 8
7.5.1 Increased Processes and N (24 proc., NB=256, N=52000) . . . 8
7.5.2 Block Size Sweep (48 proc., N=60000, NB=256 and NB=384) . . . 8

8 Performance Analysis **8**
8.1 Complete Results . . . 8
8.2 Strong Scaling: Speedup and Efficiency . . . 8
8.3 Fundamental Limits: Shared Memory vs Distributed Memory . . . 10
8.4 Discussion . . . 12
8.4.1 Phase 1 – Strong Scaling. . . . 12
8.4.2 Phase 2 – Parameter Sensitivity. . . . 12

9 Conclusion **13**

1 Introduction

This project focuses on the design, implementation, and performance evaluation of a distributed-memory computing cluster built from commodity hardware. The primary objective is to move beyond traditional sequential computing models and engage directly with the complexities of parallel and distributed systems.

The cluster was constructed using three Lenovo ThinkStation P340 desktops as compute nodes provided by the course instructor, and a personal laptop acting as the head node to manage jobs in the cluster. All machines were connected through a dedicated private Ethernet network configured with static IP addressing, passwordless SSH authentication, a shared filesystem via NFS, and a parallel computing environment built on OpenMPI.

The cluster is a wordplay, *Tricolor*, reflecting both Venezuelan national identity and the Trinity College context. Its three compute nodes are named *Araguaney*, *Turpial*, and *Orquidea*, which are the Venezuelan national symbols: the national tree, the national bird, and the national flower, respectively.

The benchmarking strategy was deliberately structured in two phases: a **strong scaling study** evaluating how performance scales with node count under a fixed workload, followed by a **parameter tuning study** exploring the sensitivity of peak performance to HPL input parameters at full scale. Together, these phases provide a comprehensive characterisation of the cluster's capacity to perform rigorous tasks and fundamental limitations, including a deep analysis of the theoretical boundaries imposed by the distributed-memory model.

2 System Architecture

2.1 Memory Models: From UMA to Distributed NUMA

To understand the performance characteristics of the Tricolor cluster, it is essential to situate it within the broader landscape of parallel computing memory models. Modern parallel systems are broadly classified according to how processors access memory: the two dominant paradigms are *shared-memory* (UMA) and *distributed-memory* (NUMA) architectures [2].

Shared Memory (UMA). In a Uniform Memory Access (UMA) system, all processor cores share a single physical memory pool via a common bus or interconnect, such that any core can read or write any memory location with equal, symmetric latency [6]. This model simplifies parallel programming considerably, as threads communicate implicitly through shared variables rather than explicit message passing. However, UMA systems introduce their own bottlenecks as core counts scale: memory bandwidth contention, cache coherence traffic, and bus saturation all impose practical ceilings on performance [8]. Furthermore, UMA systems are inherently bounded by the physical memory and core count of a single machine. The Intel Core i7-10700 in each ThinkStation compute node is itself a UMA system: all 8 physical cores share 16 GB of DDR4 RAM via a single integrated memory controller operating at a peak bandwidth of approximately 45.8 GB/s [7].

Distributed Memory (NUMA). A distributed-memory cluster extends parallelism beyond the single-node limit by connecting independent UMA nodes via a network interconnect [4]. Each node has exclusive access to its own local memory; there is no shared address space across nodes. Coordination between nodes requires explicit message passing, typically implemented via the Message Passing Interface (MPI) standard [9]. This introduces network latency and bandwidth as

fundamental constraints: whereas intra-node cache-to-cache communication occurs at nanosecond timescales, inter-node communication over commodity Ethernet operates at latencies two to three orders of magnitude higher [10]. This asymmetry is what defines Non-Uniform Memory Access (NUMA) at the cluster level.

The Tricolor cluster operates as a distributed-memory system. Each ThinkStation node is an independent UMA system; together, they form a distributed NUMA cluster where inter-node communication is mediated by a 1 GbE Ethernet switch. This architecture enables the cluster to exceed the memory and compute capacity of any single node, but imposes a fundamental communication overhead that limits parallel efficiency. This is more formalised by Amdahl's Law [1] and directly observable in the benchmarking results of Section 8.

2.2 Hardware Components

The system consists of four main components:

- **Head Node (Laptop):** Control and launch unit for job submission and SSH-based communication. An M1 MacBook Air 2020 with 16 GB of memory, used exclusively for cluster management.
- **Compute Nodes:** Three Lenovo ThinkStation P340 SFF desktops, each equipped with an Intel Core i7-10700 CPU (8 cores / 16 threads at 2.90 GHz) and 16 GB DDR4 RAM.
- **Network Switch:** Netgear 5-port switch for a dedicated private Ethernet network.
- **Ethernet Connections:** Cat5e/Cat6 cables, with a USB-to-Ethernet adaptor on the laptop.

2.3 Hardware Specifications

Specification	Value
CPU Model	Intel Core i7-10700
Cores per Node	8 physical / 16 logical (HT)
Clock Speed	2.90 GHz (base)
RAM per Node	16 GB DDR4
Total Nodes	3 compute + 1 head
Total CPU Cores	24 physical / 48 logical
Total RAM	48 GB
Network	1 GbE via Netgear switch

Table 1: Hardware specifications of each compute node.

3 Implementation

3.1 Operating System Installation

Each ThinkStation was freshly rebooted by entering the system's BIO, booting a USB flash drive with Ubuntu 22.04 LTS to eliminate pre-existing restrictions and ensure full system control. An identical user account (`liu`) was created on each node with uniform home directory structures.

The head node (laptop) ran macOS and was used exclusively for cluster management; MPI jobs were always launched from within the compute nodes themselves, as the macOS MPI installation is incompatible with the Ubuntu OpenMPI build on the nodes.

3.2 Operational Modes

- **Internet Mode:** Temporary DHCP configuration for package installation.
- **Cluster Mode:** Static IP configuration for isolated internal communication on the 192.168.1.0/24 subnet.

4 Networking and Configuration

4.1 Static IP Assignment

Node	Role	IP Address
Laptop	Head Node	192.168.1.10
Araguaney	Compute Node 1	192.168.1.101
Turpial	Compute Node 2	192.168.1.102
Orquidea	Compute Node 3	192.168.1.103

Table 2: Static IP address assignments for all cluster nodes.

4.2 Passwordless SSH

Ed25519 key pairs were generated on the head node and distributed to all compute nodes via `ssh-copy-id`. The process was repeated on each compute node to enable peer-to-peer MPI spawning, which is required for inter-node job launching.

```
ssh-keygen -t ed25519
ssh-copy-id liu@192.168.1.101
ssh-copy-id liu@192.168.1.102
ssh-copy-id liu@192.168.1.103
```

Listing 1: Passwordless SSH setup on Head node

4.3 NFS Shared Filesystem

The head node exports a centralised directory mounted at `/home/liu/cluster` on all compute nodes. HPL binaries compiled once are immediately accessible cluster-wide. `soft` mount semantics with a 3-second timeout prevent indefinite hangs on network failure:

```
192.168.1.10:/Users/Liu/cluster  /home/liu/cluster  nfs soft,timeo=30,vers=3  0
    0
```

Listing 2: /etc/fstab NFS entry on each node

Then, to validate that the shared file was working, from each node, a text file was created, and then, from each node, the shared directory was checked to see that each node's text file was successfully created.

5 Software Stack

Component	Package / Version
Operating System	Ubuntu Server 22.04 LTS
MPI Implementation	OpenMPI 4.1.6
C Compiler	GCC (system default)
Fortran Compiler	GFortran
BLAS Library	OpenBLAS 0.3.26 (pthread)
Build System	GNU Make
Benchmark	HPL 2.3

Table 3: Software stack installed on all compute nodes.

OpenBLAS was installed for benchmarking as it ships with hand-optimised kernels for modern Intel microarchitectures and consistently outperforms the generic ATLAS build in practice. HPL was therefore linked against OpenBLAS for all benchmarking runs. GFortran was installed as a required dependency of the HPL build system, which references Fortran runtime libraries during linking. HPL 2.3 was compiled from source on the NFS shared directory, making `xhpl` immediately available to all nodes.

6 Testing and Verification

System functionality was verified through a series of incremental tests before proceeding to benchmarking.

6.1 Network Connectivity

ICMP ping tests confirmed zero packet loss to all three nodes, validating static IP assignment and network integrity.

6.2 MPI Hostname Test

```
mpirun -np 3 \
  --host 192.168.1.101:4,192.168.1.102:4,192.168.1.103:4 \
  hostname
```

Listing 3: MPI hostname test

Output:

```
araguaney
turpial
orquidea
```

All three hostnames were returned simultaneously, confirming MPI can successfully SSH into all nodes and spawn remote processes. This was run multiple times, so the output was always changing its order.

6.3 MPI Hello World Programme

A standard MPI verification program was written and executed across all three compute nodes. This also serves as the live parallel job demonstration for this project. As noted above, the head node (laptop) could not participate directly in MPI execution due to macOS incompatibility with the Ubuntu OpenMPI build, and the machine's restrictions[1] prevented installing a compatible Linux environment. The three compute nodes therefore constitute the active parallel environment, with the head node serving exclusively as the job controller.

```c
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int world_size, world_rank;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    char hostname[256]; int hostname_len;
    MPI_Get_processor_name(hostname, &hostname_len);
    printf("Hello␣from␣rank␣%d␣of␣%d␣on␣host␣%s\n",
           world_rank, world_size, hostname);
    MPI_Finalize();
    return 0;
}
```

Listing 4: MPI verification programme

```
mpicc -o hello_mpi hello_mpi.c
mpirun -np 3 \
  --host 192.168.1.101:1,192.168.1.102:1,192.168.1.103:1 \
  ./hello_mpi
```

Listing 5: Compilation and execution across all three nodes

Output:

```
Hello from rank 0 of 3 on host araguaney
Hello from rank 1 of 3 on host turpial
Hello from rank 2 of 3 on host orquidea
```

Each process correctly reported its rank, total process count, and host machine. This confirmed that MPI, SSH key distribution, and the NFS shared filesystem were all functioning correctly as an integrated system, and constitutes a successful live demonstration of a parallel job running concurrently across all active compute nodes.

6.4 NFS Verification

A test file written on the head node was immediately visible on all three compute nodes, confirming correct NFS mount and synchronisation. This process was already explained previously in Section 4.3.

[1] As I do not own a personal computer, I utilise a university-loaned machine. Hence, I had limited privileges on the system's level.

7 Benchmarking

7.1 HPL Overview

The High-Performance Linpack (HPL) benchmark solves a dense $N \times N$ linear system via LU factorisation with partial pivoting, reporting performance in GFLOPS. It is the standard benchmark used to rank the TOP500 supercomputers. Key parameters:

- **N**: Matrix order. Should consume ≈80% of total RAM.
- **NB**: Block size. Controls cache utilisation; larger values increase arithmetic intensity per synchronisation point.
- **P**×**Q**: Process grid, must equal total MPI ranks.

7.2 Benchmarking Strategy

The benchmarking was conducted in two deliberate phases:

Phase 1 – Strong Scaling Study. Parameters were held constant (N = 40,000, NB= 232, 4 processes per node) whilst the number of active nodes was varied from 1 to 3. This isolates the effect of adding hardware on a fixed workload, directly revealing communication overhead and its growth with node count. The results feed directly into speedup and efficiency calculations that quantify the gap between measured and ideal parallel performance.

Phase 2 – Parameter Tuning Study. With the scaling profile established, the node count was fixed at 3 and two key parameters were varied: MPI processes per node ($4 \to 8 \to 16$) and block size NB ($232 \to 256 \to 384$), whilst also increasing N to better utilise available memory. This phase isolates the effect of algorithmic choices on performance at full hardware scale.

This two-phase structure mirrors established HPC methodology: verify scaling behaviour first, then optimise the configuration for peak throughput.

7.3 HPL Build Configuration

```
TOPdir   = /home/liu/cluster/hpl-2.3
LAdir    = /usr/lib/x86_64-linux-gnu/openblas-pthread
LAlib    = $(LAdir)/libopenblas.a
CC       = mpicc
CCFLAGS  = $(HPL_DEFS) -fomit-frame-pointer -O3 -funroll-loops
```

Listing 6: Key entries in Make.tricolor

7.4 Phase 1: Strong Scaling

7.4.1 1-Node (P=2, Q=2)

```
mpirun -np 4 --host 192.168.1.103:4 ./xhpl
```

WR11C2R4	40000	232	2	2	348.33 s	122.50 GFLOPS	*PASSED*

7.4.2 2-Node (P=2, Q=4)

```
mpirun -np 8 --host 192.168.1.101:4,192.168.1.102:4 ./xhpl
```

WR11C2R4	40000	232	2	4	232.82 s	183.27 GFLOPS	*PASSED*

7.4.3 3-Node (P=3, Q=4)

```
mpirun -np 12 \
  --host 192.168.1.101:4,192.168.1.102:4,192.168.1.103:4 \
  ./xhpl
```

WR11C2R4	40000	232	3	4	183.68 s	232.30 GFLOPS	*PASSED*

7.5 Phase 2: Parameter Tuning

7.5.1 Increased Processes and N (24 proc., NB=256, N=52000)

WR11C2R4	52000	256	4	6	399.89 s	234.42 GFLOPS	*PASSED*

Only a marginal gain (+0.9%) despite doubling the process count, signalling that communication overhead was beginning to dominate.

7.5.2 Block Size Sweep (48 proc., N=60000, NB=256 and NB=384)

WR12R2R2	60000	256	6	8	658.94 s	218.54 GFLOPS	*PASSED*

WR12R2R2	60000	384	6	8	595.03 s	242.01 GFLOPS	*PASSED*	**[Peak]**

NB= 384 produced the cluster peak of **242.01 GFLOPS**. The attempted $N = 70{,}000$ run was aborted by a bus error (signal 7), consistent with memory exhaustion: $70{,}000^2 \times 8$ bytes$/3 \approx 13.1$ GB per node, saturating physical RAM.

All runs passed the HPL residual correctness check, described by Eq. (1).

$$\frac{\|Ax - b\|_\infty}{\varepsilon \cdot (\|A\|_\infty \cdot \|x\|_\infty + \|b\|_\infty) \cdot N} < 16.0 \tag{1}$$

8 Performance Analysis

8.1 Complete Results

8.2 Strong Scaling: Speedup and Efficiency

The speedup and efficiency were measured with respect to the GFLOPS/s instead of times, as the cluster's response is what is being studied. Those two measures are calculated as shown by (2)

Phase	Nodes	Proc.	N	NB	Time (s)	GFLOPS
1	1	4	40,000	232	348.33	122.50
1	2	8	40,000	232	232.82	183.27
1	3	12	40,000	232	183.68	232.30
2	3	24	52,000	256	399.89	234.42
2	3	48	60,000	256	658.94	218.54
2	3	48	60,000	384	595.03	**242.01**

Table 4: HPL benchmark results across all Phase 1 and Phase 2 runs.

and (3). Then, the results are tabulated for all nodes. Those results are plotted in Fig. 1 and 2, showing the ideal trend and the measured one.

$$S(n) = \frac{\text{GFLOPS}(n)}{\text{GFLOPS}(1)} \tag{2}$$

$$E(n) = \frac{S(n)}{n} \times 100\% \tag{3}$$

Nodes	GFLOPS	Speedup	Efficiency
1	122.50	1.00×	100.0%
2	183.27	1.50×	74.8%
3	232.30	1.90×	63.2%

Table 5: Parallel speedup and efficiency for the Phase 1 strong scaling study.

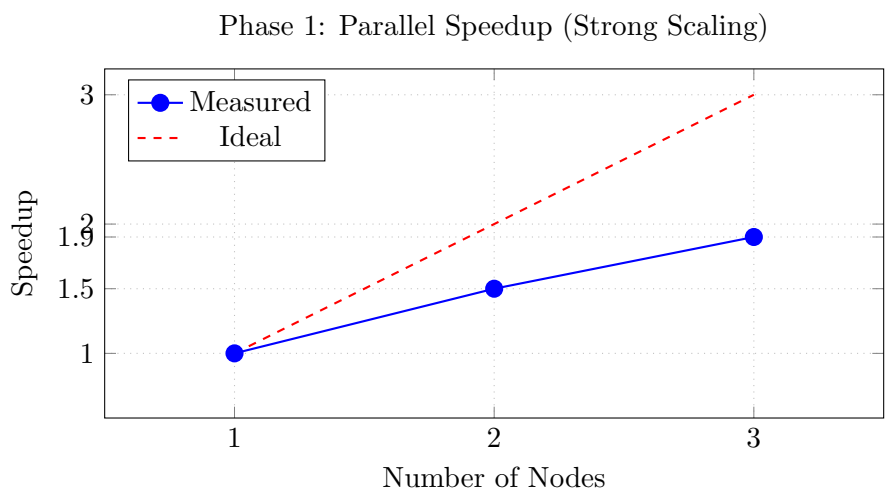

Figure 1: Measured vs. ideal speedup. The widening gap reflects growing inter-node communication overhead.

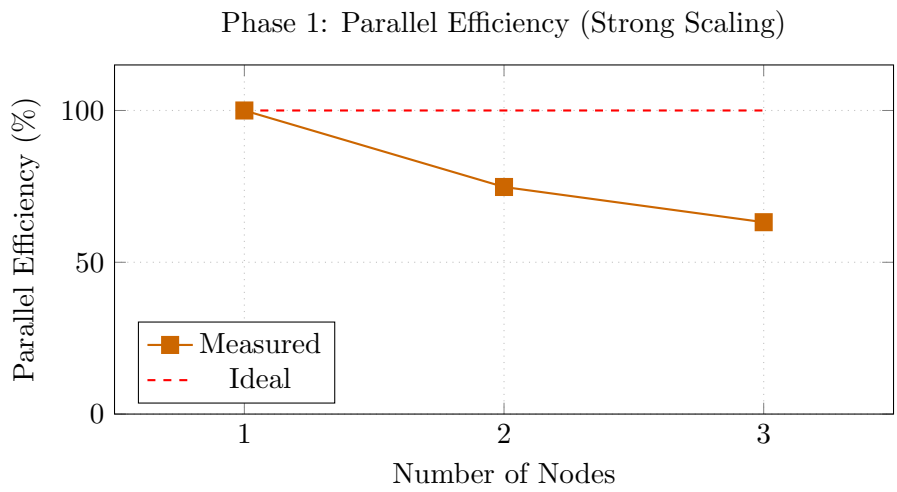

Figure 2: Efficiency declines monotonically, characteristic of fixed-size distributed workloads on commodity interconnects.

8.3 Fundamental Limits: Shared Memory vs Distributed Memory

The benchmarking results are empirical observations that reflect deep theoretical limits that distinguish the shared-memory and distributed-memory computing models.

The Shared-Memory Ceiling. Within a single node, the i7-10700 represents a UMA shared-memory system. All 8 cores access 16 GB of RAM through a single memory controller, communicating via the CPU's internal cache hierarchy (L1/L2/L3) at nanosecond latencies. The theoretical peak of a single node, assuming perfect cache utilisation and no memory bandwidth contention, is bounded by:

$$\text{Peak}_{\text{node}} = \text{cores} \times \text{clock} \times \text{FLOPs/cycle} = 8 \times 2.9\,\text{GHz} \times 16 = 371.2\,\text{GFLOPS}$$

(assuming AVX2 with FMA: 2×8 doubles per cycle = 16 FLOPs/cycle.) The measured single-node result of 122.50 GFLOPS represents approximately 33% of this theoretical peak, a realistic efficiency for a generic BLAS workload without AVX-specific tuning.

The shared-memory model's fundamental limit is memory bandwidth: as more cores simultaneously access data, the memory bus becomes a bottleneck. This is why intra-node scaling (adding more MPI processes per node) beyond the physical core count yields diminishing returns, as observed in the tuning phase.

The Distributed-Memory Communication Wall. When computation is distributed across multiple nodes, the fundamental limit shifts from memory bandwidth to network latency and bandwidth. Every inter-node data exchange incurs:

$$t_{\text{comm}} = t_{\text{latency}} + \frac{\text{message size}}{B_{\text{network}}}$$

where t_{latency} on a 1 GbE Ethernet network is typically 100–200 μs (vs. $\sim$100 ns for shared cache access—a factor of $\sim$1000$\times$ slower). HPL's LU factorisation requires $O(N^2)$ communication operations during panel broadcasts and row swaps, meaning this penalty is paid repeatedly throughout the computation.

Amdahl's Law predicts that for a fixed serial fraction f, the maximum achievable speedup is bounded by:

$$S_{\max} = \lim_{n\to\infty} \frac{1}{(1-f) + \frac{f}{n}} = \frac{1}{f} \tag{4}$$

However, in a distributed-memory system such as Tricolor, the communication overhead is not a fixed fraction of the workload, as it grows with the number of nodes, considering that each additional node introduces more collective MPI operations. The measured speedup of $S(3) = 1.90\times$ against an ideal of $3.00\times$ shows a parallel efficiency of 63.2%, and the declining efficiency from 74.8% at 2 nodes to 63.2% at 3 nodes confirms that the effective serial fraction is itself growing with n. This is characteristic of communication-dominated scaling and means the system would reach a performance ceiling well below linear speedup at larger node counts.

The Tuning Phase as Empirical Evidence. The parameter tuning results provide direct empirical evidence of these limits. Doubling the process count from 12 to 24 yielded only +0.9% improvement, because the additional inter-process communication overhead (now both intra- and inter-node) nearly cancelled the computational gain. At 48 processes with NB= 256, performance actually degraded below the 12-process baseline, thus demonstrating that the communication wall had been crossed. Increasing NB to 384 recovered +10.7% by reducing the number of MPI synchronisation points per unit of computation, effectively pushing the computation-to-communication ratio back in favour of computation. This is the same principle that motivates the use of high-latency-tolerant algorithms in distributed HPC: maximise local computation between communication events.

Gustafson's Law and Weak Scaling. Amdahl's Law assumes a fixed problem size, which inherently penalises distributed systems as the serial fraction becomes dominant. Gustafson's Law [5] offers a complementary perspective: if problem size is allowed to grow with the number of nodes, the parallel portion dominates and the scaled speedup becomes:

$$S_G(n) = n - f(n-1) \tag{5}$$

The benchmarking strategy employed in Phase 2 implicitly reflects this model: as node count was held at 3, the matrix size N was progressively increased from 40,000 to 60,000, exploiting the additional aggregate memory. The resulting peak of 242.01 GFLOPS represents a 97.6% improvement in throughput over the single-node baseline whilst solving a 50% larger problem, which is consistent with the weak-scaling argument that distributed systems are best evaluated by the *amount of useful work performed*, not solely by time-to-solution on a fixed workload.

The Karp-Flatt Metric: Empirical Serial Fraction. A limitation of Amdahl's Law is its assumption that the serial fraction f is constant across node counts. In practice, communication overhead grows with n, making f itself a function of n. The Flatt metric [3] addresses this by computing an empirical apparent serial fraction at each measured data point:

$$e(n) = \frac{\dfrac{1}{S(n)} - \dfrac{1}{n}}{1 - \dfrac{1}{n}} \tag{6}$$

Applied to the Phase 1 results:

Nodes	**Speedup** $S(n)$	**Efficiency**	e
1	1.00×	100.0%	—
2	1.50×	74.8%	0.333
3	1.90×	63.2%	0.290

Table 6: Karp-Flatt empirical serial fraction computed from Phase 1 results.

The Karp-Flatt metric declines from $f_e = 0.333$ at 2 nodes to $f_e = 0.290$ at 3 nodes. A declining f_e with increasing n indicates that the apparent serial fraction is not growing as nodes are added which, under Amdahl's fixed-f assumption, would suggest the system is scaling reasonably well and is not yet fully communication-dominated. However, this interpretation must be treated cautiously for two reasons.

First, this metric is a diagnostic tool that quantifies how much of the workload behaves serially, but it cannot distinguish why: a high or growing e could reflect genuine algorithmic serial sections, MPI synchronisation overhead, load imbalance, or memory bandwidth saturation, all of which produce identical signatures in the metric. With only two data points ($n = 2$ and $n = 3$), it is statistically impossible to determine whether e is genuinely declining, plateauing, or about to rise sharply at $n = 4$ or beyond.

Second, the small node count means each data point carries significant weight, and any measurement noise, such as background OS processes or NFS latency fluctuations, directly distorts the computed e. A meaningful Karp-Flatt analysis would require at least five to six node counts to identify a reliable trend.

The results should therefore be interpreted conservatively: the metric confirms that the cluster is operating in a regime where parallelism still provides measurable benefit, but it cannot definitively diagnose the root cause of the sub-linear scaling observed.

8.4 Discussion

8.4.1 Phase 1 – Strong Scaling.

The scaling results follow the pattern predicted by Amdahl's Law: speedup is real but sub-linear, with efficiency declining from 100% at 1 node to 63.2% at 3. The primary cause is the 1 GbE interconnect, which introduces millisecond-scale latency per collective MPI operation. Which is, in fact, approximately 1000× slower than intra-node cache communication. Hence, this benchmarking phase helped diagnose that the Ethernet switch is the principal architectural bottleneck of the Tricolor system.

8.4.2 Phase 2 – Parameter Sensitivity.

The tuning phase revealed two critical results. First, increasing process count without compensating for communication overhead is counter-productive: 48 processes at NB= 256 underperformed the 12-process baseline. Second, block size is the single most impactful tuning parameter: NB= 384 improved throughput by 10.7% over NB= 256 at identical hardware, by better utilising the i7-10700's 16 MB L3 cache and reducing collective synchronisation frequency.

The peak of **242.01 GFLOPS** represents a 97.6% improvement over the single-node baseline, demonstrating that meaningful distributed performance requires co-optimisation of both hardware utilisation and algorithmic parameters.

9 Conclusion

This project successfully demonstrated the design, construction, and operation of a functional distributed HPC cluster from commodity hardware. The two-phase benchmarking strategy produced a rigorous and comprehensive characterisation of the performance of the system.

The cluster achieved a peak of **242.01 GFLOPS** (N = 60,000, NB= 384, 48 processes, 3 nodes), a 97.6% improvement over the single-node baseline of 122.50 GFLOPS. The parallel efficiencies of 100%, 74.8%, and 63.2% from the scaling study are consistent with Amdahl's Law and reflect the fundamental communication penalty of a 1 GbE distributed-memory architecture. The tuning phase directly demonstrated the transition from computation-bound to communication-bound operation: over-provisioning MPI processes without appropriate block size adjustment degraded rather than improved performance, validating the theoretical analysis of Section 8.3.

The project made concrete and measurable the core tension of distributed NUMA computing: network latency imposes a communication wall that no amount of additional hardware can overcome without algorithmic adaptation. This wall is absent in the shared-memory UMA model but becomes the dominant engineering constraint the moment computation crosses a node boundary. This is a lesson demonstrated quantitatively by every benchmarking run in this report.

Future work could include: compiling ATLAS from source for genuine auto-tuning and exploring different frameworks for cluster benchmarking; weak scaling analysis with N growing proportionally to node count; utilising SLURM for resource management; and upgrading the interconnect to InfiniBand or 10 GbE to reduce the communication bottleneck identified throughout both benchmarking phases.

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the Spring Joint Computer Conference (AFIPS)*, pages 483–485. ACM, 1967.

[2] David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. *Parallel Computer Architecture: A Hardware/Software Approach*. Morgan Kaufmann, 1999.

[3] Howard P. Flatt and Ken Kennedy. Performance of parallel processors. *Parallel Computing*, 12(1):1–20, 1989.

[4] William Gropp, Ewing Lusk, and Anthony Skjellum. *Using MPI: Portable Parallel Programming with the Message Passing Interface*. MIT Press, 2nd edition, 1999.

[5] John L. Gustafson. Reevaluating Amdahl's law. *Communications of the ACM*, 31(5):532–533, 1988.

[6] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.

[7] Intel Corporation. *Intel® Core™ i7-10700 Processor Specifications*. Intel Corporation, 2020. Accessed: April 2026.

[8] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE Technical Committee on Computer Architecture Newsletter*, pages 19–25, December 1995.

[9] MPI Forum. MPI: A message-passing interface standard, version 4.1. Technical report, MPI Forum, 2023. Accessed: April 2026.

[10] TOP500 Project. TOP500 supercomputer sites, 2024. Accessed: April 2026.